
Weave of Formal Thought

Alexandre Bouayad
G-Research
alex.bouayad@gmail.com

TECHNICAL REPORT

Abstract

Large language models (LLMs) attain remarkable surface fluency on code, yet they do not formally guarantee the syntactic validity of their output, nor do they typically leverage the hierarchical structure that defines the target language. While existing constrained-decoding frameworks (e.g., XGrammar, Guidance, Outlines, SynCode) offer a solution to the former, they predominantly operate under rigid assumptions that preclude critical lexical mechanisms relied upon by modern parsers — including context-sensitive lexing (e.g., Pythonic indentation), maximal-munch tokenization, and keyword extraction — and only approximate the masking of invalid subword tokens, sacrificing completeness. For the latter, modern code LLMs inject grammatical structure during training through predetermined policies rather than learning which structural information to expose. In this work, we introduce *Weave of Formal Thought* (WoFT), an overarching paradigm that unites rigorous syntactic validation with learned structural representations. First, we present a formal engine and constrained decoder that is sound and complete with respect to the full Tree-sitter specification by augmenting generalized LR (GLR) parsing with a novel *speculative-lexing* construction that maintains concurrent lexer-state hypotheses synchronized with the GLR graph-structured stack; the decoder admits every subword token that extends to a valid program prefix and rejects every token that does not. Second, we present a latent-variable fine-tuning method that trains the language model to interleave non-terminal grammar symbols directly into the generation process. Utilizing the reweighted wake-sleep (RWS) algorithm to optimize the importance-weighted evidence lower bound (IW-ELBO) of the surface text, the model learns to selectively retain formal derivations as an adaptive structural scratchpad. For Python, fine-tuning StarCoder2-3B with our RWS objective reduces per-token cross-entropy by 14.3% relative to a text-only SFT baseline, demonstrating that discretionary latent syntax recovers critical structural information that flat autoregressive training discards. Our code and implementation are publicly available at <https://github.com/alexbouayad/formal>.

1 Introduction

Modern autoregressive large language models (LLMs) trained on code attain remarkable surface fluency: they reproduce idiomatic syntax, identifier conventions, and library-specific patterns with such fidelity that the generated text is, on average, statistically indistinguishable from human-written programs. Yet this competence is brittle in two distinct senses. First, fluency is not formal correctness: even a state-of-the-art code model produces output that fails to parse, type-check, or compile at non-trivial rates, necessitating downstream filtering or repair pipelines whose cost grows with the strictness of the target specification. Second, and more fundamentally, the model has no *explicit*

access to the hierarchical structure that *defines* the language. Every program admits a unique, finite, and mathematically rigorous abstract syntax tree (AST) derivation; this derivation is precisely the information that would let the model plan a function before writing its body, allocate a loop variable before its first use, or close a scope before opening a new one. Standard autoregressive training, by collapsing the joint distribution of text and structure onto the marginal over text alone, forces the model to *re-discover* this hierarchy implicitly at every forward pass.

Recent attempts to inject explicit reasoning into language models have largely proceeded in the opposite direction. Chain-of-thought (CoT) prompting [1] and its internal variants—Pause Tokens [2], Quiet-STaR [3], and continuous-latent reasoning [4]—grant the model additional intermediate compute, but the reasoning trajectory itself is either free-form natural language or unconstrained latent vectors. Neither offers a verifier that can reject ill-formed reasoning steps, and neither aligns with the discrete, hierarchical structure that programming languages already provide. The semantic planning gap is thus a neuro-symbolic gap: the structure of the target language exists, formally and unambiguously, but is exploited neither at training time nor at decoding time.

A parallel line of work—constrained decoding—attacks the correctness side of the problem by masking the vocabulary at each step to the tokens consistent with a partial parse [5, 6, 7, 8, 9, 10, 11]. These engines have driven structured generation latency close to that of unconstrained decoding, but their formal coverage has lagged behind the languages practitioners actually wish to generate.

To bridge this neuro-symbolic gap, we introduce *Weave of Formal Thought* (WoFT), a unified paradigm that integrates rigorous syntactic validation with learned structural representations. WoFT operates through two complementary components: a formal inference engine (metaphorically, the “loom”) and a latent-variable fine-tuning method (the “weaver”).

First, we present a language-model-agnostic formal engine and constrained decoder that is sound and complete with respect to the full Tree-sitter specification [12]. By augmenting Generalized LR (GLR) parsing [13] with a speculative-lexing construction, our engine maintains concurrent lexer-state hypotheses synchronized with a GLR graph-structured stack. This approach natively supports context-sensitive lexing (such as Pythonic indentation), maximal-munch tokenization, and declarative ambiguity resolution. The resulting decoder admits every subword token that extends to a valid program prefix and rejects every token that does not.

Second, we introduce a latent-variable fine-tuning method that trains the language model to interleave non-terminal grammar symbols directly into the generation process. Rather than forcing the model to emit a fixed, deterministic syntax trace, we treat formal derivations as discrete latent variables. We train the model using the reweighted wake-sleep (RWS) algorithm [14] to optimize the importance-weighted evidence lower bound (IW-ELBO) [15] of the surface text. This enables the model to utilize formal non-terminals as an adaptive structural scratchpad, retaining them only when they effectively compress future surface tokens. Empirically, fine-tuning StarCoder2-3B [16] on Python with our RWS objective yields a 14.3% relative reduction in surface-token cross-entropy compared to a standard text-only SFT baseline, demonstrating that discretionary latent syntax recovers critical structural information that flat autoregressive training discards. Our complete implementation, including the formal engine and RWS training pipeline, is publicly available at <https://github.com/alexbouayad/formal>.

The remainder of the paper is organized as follows. Section 2 reviews related work across constrained decoding, latent structure in language modeling, and internal reasoning. Section 3 details the design and theoretical guarantees of our formal engine. Section 4 presents our latent-variable formulation, the RWS optimization objective, and the training architecture. Section 5 details our experimental setup and reports empirical results on surface token modeling. Finally, Section 6 concludes with our research vision and next steps.

2 Related Works

2.1 Constrained decoding and formal grammars

Constrained decoding intervenes during autoregressive generation to guarantee that language model outputs satisfy a formal-language specification, typically by masking the model’s vocabulary at each step to admit only tokens consistent with a partial parse. PICARD [5] introduced incremental

parser-guided decoding for SQL; Outlines [6] and SynCode [7] generalized this approach to arbitrary regular and context-free languages by compiling the grammar into finite-state machines and pushdown automata, respectively, and by precomputing per-token masks offline. Recent engines push these ideas toward production-scale latency. XGrammar [8, 9] partitions the vocabulary into context-independent and context-dependent subsets to amortize mask construction across decoding steps, layered on top of byte-level Earley [17] and pushdown-automaton recognizers. Guidance [11] and its Rust backend LLGuidance [10] fuse derivative-based parsing directly with the decoding loop.

A complementary line of work identifies the misalignment between subword vocabularies and formal language lexemes as a primary source of correctness failures. DOMINO [18] explicitly addresses this gap by combining offline precomputation with speculative decoding to enforce constraints in a subword-aligned fashion, achieving near-zero overhead on regex- and CFG-bounded targets. We share DOMINO’s diagnosis but operate one level higher in the compiler stack: rather than speculating over subword sequences within a fused regex/CFG state machine, we maintain multiple concurrent lexer states alongside a Generalized LR [13] graph-structured stack. This preserves a separate, mature lexer at the granularity of formal lexemes and supports stateful lexical behaviors that scannerless engines cannot express.

Our formal engine instead fully integrates with the Tree-sitter framework [12], retaining its full grammar specification language and its compiler-grade lexer. We resolve the subword-alignment problem by maintaining a finite set of speculative lexing paths over a GLR graph-structured stack: each path corresponds to an active lexer-state hypothesis consistent with the subword tokens emitted so far, and a vocabulary token is admitted if and only if at least one path can extend to a valid prefix in the language. Section 3 formalizes this construction; under standard well-formedness assumptions on the Tree-sitter grammar, the resulting decoder is sound and complete with respect to the full Tree-sitter specification.

2.2 Latent structure in language modeling

The integration of explicit syntactic structure into language modeling has a rich history in natural language processing. Early structured language models [19, 20, 21] demonstrated that conditioning next-token prediction on partial parse trees improved perplexity. With the advent of deep learning, frameworks like Recurrent Neural Network Grammars (RNGs) [22] and latent Probabilistic Context-Free Grammars (PCFGs) [23] attempted to treat these syntactic trees as latent variables. These models optimized the marginal likelihood of the observed text by marginalizing over all possible tree structures using dynamic programming (e.g., the Inside-Outside algorithm).

While theoretically elegant, these historical latent-structure models faced computational scaling limits. Exact marginalization over parse trees scales cubically ($\mathcal{O}(N^3)$) with sequence length, making it computationally intractable for the large vocabularies and context windows of modern Transformers. Consequently, the standard language modeling paradigm abandoned explicit latent syntactic structures, relying entirely on self-attention mechanisms to implicitly learn flat representations of code and text.

More recent work has pushed internal reasoning further into latent space. Coconut [4] replaces discrete chain-of-thought tokens with continuous hidden-state vectors fed back as input embeddings, allowing the model to encode multiple alternative reasoning steps simultaneously and perform breadth-first search over reasoning paths. While this circumvents the textual-coherence overhead of natural-language reasoning, the resulting latents are continuous vectors with no symbolic semantics; they cannot be validated, interpreted, or composed by an external verifier. Our framework occupies a complementary point in the design space: latents are discrete non-terminal symbols drawn from a formal grammar, and every latent trajectory is verified against a Tree-sitter GLR oracle. This trades Coconut’s continuous geometry for symbolic groundedness, hierarchical interpretability, and the ability to mathematically constrain the reasoning trajectory to valid AST derivations.

We modernize the historical objective of latent-structure language modeling for autoregressive Transformers. Instead of attempting exact marginalization over a vast subword space, we leverage Tree-sitter as a symbolic oracle. By modeling formal abstract syntax tree (AST) derivations as discrete latent variables trained via the reweighted wake-sleep (RWS) algorithm, our model learns to optimize the importance-weighted evidence lower bound (IW-ELBO) of terminal sequences. This allows the language model to explicitly leverage hierarchical syntax without the $\mathcal{O}(N^3)$ bottleneck of earlier structured models.

2.3 Internal chain-of-thought and reasoning

Standard chain-of-thought (CoT) prompting [1] improves the semantic planning of language models by allocating additional intermediate compute steps (forward passes) before yielding a final answer. Recently, there has been a push toward internal or implicit reasoning—allowing models to “think” in a hidden scratchpad that is omitted from the final output. Techniques like Pause Tokens [2] insert dummy tokens to delay generation, while Quiet-STaR [3] trains models via REINFORCE to generate hidden natural language rationales, rewarding internal thoughts that minimize the cross-entropy of future target tokens.

Current internal reasoning frameworks rely heavily on free-form, unconstrained natural language. Because the hidden rationales are generated autoregressively without formal boundaries, they remain susceptible to semantic drift, hallucination, and an unguided search space. When writing code, unconstrained internal thoughts often fail to align with the strict topological requirements of the target algorithm.

Our framework redefines internal reasoning by replacing unconstrained natural language with strict formal grammars. By selectively retaining non-terminal symbols (e.g., `<statement>`, `</statement>`) via our proposal head, the language model engages in latent syntactic planning. These non-terminals act as a rigorously structured internal chain-of-thought. Rather than generating unconstrained text rationales, the model explores latent structural derivations derived from the true abstract syntax tree of the code. While future work may employ the formal engine to mathematically constrain and filter valid AST derivation paths during active reasoning, our current variational formulation directly grounds the internal reasoning process by learning to selectively route through valid syntactic blueprints. This provides the model with explicit hierarchical structure that improves semantic planning and surface modeling while avoiding the semantic drift of ungrounded CoT.

3 The Formal Engine

The first component of the Weave of Formal Thought (WoFT) paradigm is the formal engine, which serves as the rigorous structural mechanism—metaphorically, the “loom”—that guarantees syntactic validity during generation. To successfully weave formal grammar tokens and surface text without tangling or breaking the underlying syntactic constraints, the inference engine must precisely track the grammatical state of the program. In this section, we describe the design of our formal engine, a language-model-agnostic constrained decoder that achieves soundness and completeness with respect to the full Tree-sitter specification.

3.1 GLR parsing and Tree-sitter

Deterministic LR parsers [24] are limited to unambiguous context-free grammars. When parsing non-deterministic grammars with shift/reduce or reduce/reduce conflicts, Generalized LR (GLR) parsing [13] bifurcates the parse stack to explore conflicting grammatical hypotheses in parallel, merging stacks when they reconverge or terminating them when they prove invalid.

Modern programming languages such as Python, JavaScript, and C++ require parsing mechanisms beyond pure context-free formalisms. These include lexical context sensitivity (e.g., disambiguating keywords from identifiers based on parse state), maximal-munch tokenization, and complex external lexing rules like indentation tracking. Modern parsers and parser generators integrate these advanced lexical mechanisms alongside GLR parsing to handle production source code.

Interactive software environments further require real-time, incremental parsing to maintain syntax trees during live editing [25]. Tree-sitter [12] provides an incremental GLR parsing infrastructure that rapidly updates syntax trees as code is modified.

However, while Tree-sitter supports incremental parsing for complete lexical edits in a live editor, language models generate code autoregressively via arbitrary subword tokens that split lexical boundaries. Because Tree-sitter’s underlying lexer and external scanners rely on complete token streams to enforce maximal-munch tokenization, they cannot evaluate whether an incomplete subword token represents a valid prefix of a future legal statement. To serve as an online constrained decoder, our formal engine extends Tree-sitter’s GLR framework with a speculative, subword-level execution model.

3.2 Architecture

The formal engine is structured as seven co-operating components: a stream buffer, a DFA-based lexer compiled from the Tree-sitter grammar, a stream scanner that wraps a Tree-sitter external scanner, a graph-structured stack, a parse lattice, an extended GLR parser compiled from the Tree-sitter grammar, and the frontend of the engine.

The architecture is language-model- and grammar-agnostic: the FRONTEND decodes subword tokens into characters and feeds them through the BUFFER one token at a time, and the LEXER, SCANNER, and PARSER handle the Tree-sitter grammar without modification.

BUFFER. At the upstream end of the engine, the BUFFER exposes a single-character lookahead reader over an append-only stream of characters. The `feed` operation appends a chunk of text, or an end-of-stream (EOS) sentinel, without disturbing the current read offset; this is how the engine ingests the characters of a subword token. The `validate` and `revert` operations together implement a checkpoint mechanism: once a token has been accepted by the PARSER, the BUFFER is validated and its checkpoint advanced to the current write head; if the token is rejected, the BUFFER is reverted and all characters written past the last checkpoint are discarded. The BUFFER owns the text; both the LEXER and the SCANNER operate on it through a small reader interface exposing offset, lookahead, and the at-end-of-file predicate.

LEXER. The LEXER is a deterministic finite-state recognizer driven by a single compiled transition table. It advances two DFA states in parallel, `plus` and `minus`, which share the same transition function but are seeded from distinct starting states that depend on the parsing context. The `plus` state recognizes the terminals that are valid lookaheads in the current parse state, including any keywords and the generic word terminal whenever the latter is itself admissible. The `minus` state is seeded when the word terminal is *not* a valid lookahead, and while at least one keyword is a valid lookahead, in which case it recognizes the word terminal alone. Together these two states give the engine the information it needs to reproduce Tree-sitter's keyword extraction. At every step the LEXER reads one character, advances the two DFA states, and records the most recent accepting position as a candidate endpoint, yielding maximal-munch tokenization. The LEXER's `lex` operation drives this loop from the current reader position: it steps both DFA tracks until both die or the *buffer* is exhausted, returns the last matched symbol together with the accepting position, and writes the surviving DFA states back into the LEXER's state before returning. When both DFA states accept at a shared endpoint, `plus` acceptance takes precedence over `minus` acceptance. The pair of DFA states constitutes the LEXER's state, which the PARSER can read and reset to suspend and resume lexing at subword token boundaries when the BUFFER is exhausted mid-lexeme.

SCANNER. Tree-sitter grammars routinely rely on external scanners written in C to recognize tokens that cannot be expressed as regular languages, e.g., indentation in Python, heredocs, raw strings, template literals. The engine integrates such scanners directly through a foreign function interface: it loads the compiled scanner library and exposes the Tree-sitter callback surface. Across scans, the SCANNER's state is persisted through Tree-sitter's native serialization protocol: at the end of each scan the state is serialized into a byte string, which the engine carries alongside the LEXER's state and restores at the start of the next scan. Within a single scan, however, a non-trivial challenge arises because external scanners are written under the assumption of a complete input: when the SCANNER requests the next character past the end of the currently available text, it cannot simply block. We resolve this by running each external scanner invocation inside a *scanlet*, a suspendable coroutine carrying one external scanner call, which yields control back to the PARSER when the external scanner hits the end of the BUFFER. The SCANNER's `scan` operation either resumes an existing scanlet or spawns a fresh one at the current BUFFER position, seeded from the serialized state and the set of recognizable external symbols, then switches execution into the scanlet; the scanlet runs the C scanner callback directly, yielding back to the PARSER whenever the callback requests a character past the BUFFER boundary, and returns the matched symbol together with the accepting position once the callback completes. Once additional text has been fed, the same scanlet is resumed and continues from exactly the C-side instruction at which it was suspended. Cloning a scanlet duplicates its suspended state, allowing each branching parse hypothesis to carry its own in-flight scan.

STACK. The parser operates over a graph-structured stack (GSS), the standard device for sharing structure across the parallel stacks of a GLR parser. The GSS is a directed acyclic graph, where each path corresponds to one stack. Each node of the GSS carries a parse state and a set of predecessor links. The push operation takes a parse state and embeds it into a new node that points back to the top of one of the parallel stacks. Two nodes carrying the same parse state can be merged by taking the union of their predecessor sets, so parse paths that converge on a common parse state can be represented by a single shared node, while the distinct histories stay separate. The pop operation, parameterized by a count, returns the set of nodes reachable by traversing backwards that many links, so the reduction of a grammar rule can fan out over multiple paths, without ever materializing the underlying parse forest.

LATTICE. A graph-structured stack captures alternative parse stacks, but the engine must also account for alternative lexical futures—suspended LEXER and SCANNER states that may resolve to a lexical token. The engine must additionally track the ordering constraints that maximal-munch tokenization imposes on such futures, and it must keep each lexical hypothesis synchronized with the parse stack it belongs to. This is realized by the LATTICE, which consists of a superposition of lexico-parse hypotheses arranged as a dependency graph. Each lattice vertex records an alternative parse stack, a BUFFER position, a LEXER state, and a SCANNER state. The SCANNER state is either a serialized Tree-sitter state or a live scanlet when an external scan is in flight. A vertex is keyed by a *signature* consisting of its top parse state, text position, LEXER state, and SCANNER state. Vertices sharing a parent vertex are grouped by their signatures and merged by taking the union of their parse stacks, so lexico-parse branches that reconverge never multiply. The LATTICE is accessed through a set of *heads* that point to its vertices. The focus operation designates as *visible*—the active target of lattice operations—the vertex pointed to by a head. The *superpose* operation inserts a new vertex between the visible vertex and its parent; its destructive dual, *collapse*, tears down the subtree beneath the visible vertex. The *create* operation spawns a new vertex under the same parent as the visible vertex; its destructive dual, *annihilate*, removes the visible vertex—reconnecting its children to its parent. The two constructive operations return a fresh head pointing to the new vertex, while the two destructive operations retire the head pointing to the vanished vertex.

PARSER. The PARSER is a GLR parser over the Tree-sitter parse and goto tables, where the collection of parse hypotheses—pairs of a stack version and a buffer position—is extended into the richer DAG structure of the LATTICE. Every parse step opens with a lex step that yields a single potential lookahead by dispatching, in order, to the SCANNER’s *scan* or the LEXER’s *lex* operation. If an external scan is already in flight, the scanlet is resumed; if the LEXER is in a suspended state, it is similarly resumed; otherwise the SCANNER is tried first, and, if it yields nothing, the LEXER is tried—the seed for both is taken from the *lex mode* of the current parse state. When the BUFFER is exhausted mid-lexeme—that is, when the SCANNER returns a scanlet or when the plus DFA track of the LEXER is still alive—the parser inserts a continuation vertex into the LATTICE via the *superpose* operation, carrying the in-progress scanlet or the LEXER’s state so that scanning or lexing can be resumed verbatim once more text arrives. A lexed lookahead (or the LEXER ending in a minus-only state; see below) triggers the collapse of the visible parse vertex. The parse vertex is eventually and unconditionally annihilated, and if a lookahead is present, the PARSER then drives shifts and reductions. Reductions are serialized via a deduplicating worklist, fanning out across the graph-structured stack at parse-table conflicts and merging derivations that reconverge on a common stack top; a shift creates a new parse vertex. The PARSER cycles through the LATTICE until all hypotheses are either exhausted or paused at the BUFFER boundary. The LATTICE is therefore the locus of the *speculative-lexing* mechanism: within a single parse cycle the parser may commit to a token at one endpoint while a parent hypothesis continues lexing past it, preserving the maximal munch property even when a subword token boundary falls beyond the characters currently in the BUFFER.

When the LEXER returns the word symbol from minus acceptance, the terminal is inadmissible in the current parse state; the parse node is therefore collapsed and annihilated and no parse action is triggered, integrating Tree-sitter’s keyword extraction—an identifier that merely begins with a keyword is lexed to its full extent and rejected, while a lexeme equal to a contextually valid keyword is shifted as that keyword. When the LEXER ends in a minus-only state at the BUFFER boundary, the PARSER conservatively retires the hypothesis via *collapse* and *annihilate*. This is the only over-approximation the engine makes to the filtering of subword tokens; we will later show that the

filter remains complete for most programming languages, essentially complete for the others, and explain how the approximation could be lifted.

FRONTEND. The FRONTEND integrates the six components above into the standard LLM generation loop via rejection sampling. Before each subword token is emitted, the FRONTEND snapshots the current LATTICE. The language model produces next-token logits; the FRONTEND takes the highest-scoring candidate token, decodes it into characters, feeds them to the BUFFER, and runs the PARSER. If at least one hypothesis of the LATTICE survives, the token is accepted: the BUFFER is validated and the FRONTEND advances. If every hypothesis dies, the token is rejected: the BUFFER is reverted, the LATTICE is restored from its snapshot, the offending token’s logit is set to $-\infty$, and the FRONTEND resamples from the updated distribution.

4 Weave of Formal Thought

While the formal engine provides the rigorous structural loom to ensure that language models output valid surface forms at inference time, constrained decoding alone fundamentally relies on an autoregressive policy that treats syntax as an external filter rather than an internalized representation. We hypothesize that modeling the latent syntactic derivation—the *formal thought* underlying the surface code—provides a richer training signal. To this end, the second component of the Weave of Formal Thought (WoFT) paradigm is a training method that fine-tunes a model to interleave non-terminal grammatical derivations as discrete latent variables during the generation process. Under this view, each formal token represents a thought of a *formal* nature, in the sense that the thought lives in a discretized space—or rather a collapsed projection of an internal one, continuing the quantum analogy—parameterizing the structural *form* of the language. We adopt the term *weave* to reflect that these formal thoughts are not chained sequentially prior to generation, but are instead interleaved directly into the surface text, acting as the structural weft that supports the surface code.

4.1 AST derivations as discrete latents

To explicitly model the hierarchical structure of the target language, we construct the abstract syntax tree (AST) of the surface code using a Tree-sitter parser and flatten it into a linear sequence that interleaves standard text subwords with formal grammar tokens. During a depth-first traversal of the AST, we emit formal tokens that serialize the syntactic derivation: these take the form of XML-like tags denoting non-terminal grammar rules (e.g., `<statement>`, `</statement>`) as well as syntactic fields (e.g., `[condition]`, `[/condition]`). This serialization is configurable: we can choose to include or exclude field tokens, and to emit only starting tag tokens (prefix placement), only ending tag tokens (postfix placement), or both, allowing us to flexibly control the density of the formal signal injected into the sequence.

The interleaved sequence thus comprises tokens drawn from two logically distinct vocabularies: the base text vocabulary and the formal grammar vocabulary. We treat the formal tokens as a sequence of discrete latent variables z , while the surface text tokens constitute the observed variables x . Our goal is to maximize the marginal likelihood of the surface code $p_\theta(x) = \sum_z p_\theta(x, z)$, effectively marginalizing over all possible subsets of formal tokens. During training, we represent the decision to keep or drop each formal token as an independent Bernoulli variable, yielding a sequence of binary submasks. By selectively masking formal tokens—essentially removing them from the sequence—we enable the model to explore and marginalize over different latent derivations.

4.2 Proposal and generative joint modeling

To optimize the marginal likelihood with discrete latents, we employ an amortized variational inference framework consisting of two components: a proposal (or inference) distribution $q_\phi(z | x)$ and a generative model $p_\theta(x, z)$. Rather than maintaining separate networks, both distributions are parameterized by a single pre-trained base causal language model augmented with a shared parameter-efficient fine-tuning (PEFT) adapter.

We extend the vocabulary of the base model by introducing dedicated input and output embedding layers for the formal tokens. The shared model processes the mixed sequence of text and formal

tokens to produce contextualized hidden states. From these shared representations, the architecture diverges into two distinct linear heads.

For the *proposal head* (ϕ), a one-dimensional linear projection maps the hidden states of the formal tokens to token-level logits. These logits parameterize independent Bernoulli distributions over the formal mask, dictating the probability of keeping each formal token in the sequence. For the *generative head* (θ), the standard language modeling output embeddings are used for surface text tokens, while the dedicated formal output embeddings are used for formal tokens.

By sharing the underlying base model and its PEFT adapter, and routing only through dedicated linear heads, we efficiently train both ϕ and θ with minimal memory and computational overhead.

4.3 Optimization via reweighted wake-sleep

While techniques such as the straight-through estimator (STE) [26] or continuous relaxations, e.g., Gumbel-Softmax [27, 28], are often used to circumvent the non-differentiability of discrete variables, they are fundamentally incompatible with our architecture. In our model, the discrete formal mask dictates more than just the weighting of tokens; it dictates the physical compaction of the sequence. When a formal token is dropped, all subsequent tokens shift left. Consequently, this single discrete choice not only alters the overall sequence length and the positional encodings, but also cascades through the causal attention mechanism to perturb the logits of every subsequent token. Because this constitutes discrete stochastic control flow—where the computational graph itself depends on the sampled latents [29]—continuous relaxations are inapplicable.

Instead, we optimize the importance weighted evidence lower bound (IW-ELBO) [15]. While the standard ELBO bounds the log marginal likelihood using a single sample, it often forces the proposal distribution to prematurely collapse around a single mode. The IWAE lower bound mitigates this by averaging over K independent particles:

$$\mathcal{L}_{\text{IWAE}} = \mathbb{E}_{z_1, \dots, z_K \sim q_\phi(z|x)} \left[\log \frac{1}{K} \sum_{k=1}^K \frac{p_\theta(x_k)}{q_\phi(z_k | x)} \right]$$

where x_k is the sequence with the submask applied. The IWAE objective is yielding a strictly tighter lower bound that encourages the proposal to explore multiple plausible trajectories. To optimize this objective, we employ the reweighted wake-sleep (RWS) algorithm [14], which provides a highly effective, low-variance gradient estimator specifically designed for models with discrete stochastic control flow [30, 31]. We outline one step of the training loop in algorithm 1. Notice that the IW-ELBO is not differentiated directly: instead, we perform gradient descent on the surrogate pseudo-loss $\mathcal{J}_{\text{RWS}}$. Additionally, during the wake- θ phase, we subsample a single particle proportional to its normalized importance weight rather than computing the exact weighted sum over all K particles. This particle-resampling approach significantly reduces the memory and computational footprint of the generative backward pass.

The algorithm relies strictly on the two *wake* phases (wake- θ and wake- ϕ), fully omitting the *sleep* phase present in the original wake-sleep algorithm. In standard settings, the sleep phase is usually dropped entirely because training the proposal on data “dreamed” by an untrained generative model introduces a detrimental data distribution bias [30]. While our situation is distinct since we are fine-tuning a pre-trained language model rather than training from scratch, we still omit the sleep phase because data sparsity is not a primary bottleneck in our setting. The model converges rapidly relative to the abundance of available surface code, rendering the sleep phase unnecessary for basic convergence. Nonetheless, future iterations could incorporate the sleep phase as a regularizer, driving the proposal to better approximate the generative posterior; this may lower the variance of the importance weights in the wake phase, thereby mitigating the self-normalized importance sampling (SNIS) bias inherent to the wake- ϕ update.

To appreciate our choice of the reweighted wake-sleep algorithm, it is instructive to compare it against alternative score-function estimators for discrete latent variables. Unbiased estimators such as VIMCO [32] reduce variance via leave-one-out baselines, but suffer from decaying signal-to-noise ratios as the number of particles K increases [33]. More recently, estimators like ARM [34] and DISARM [35] utilize antithetic sampling to achieve strictly lower variance for categorical variables, while OVIS [36] derives the theoretically optimal control variate. However, because these estimators scale their gradients by quantities that correlate with the magnitude of the objective function, they

Algorithm 1 Reweighted Wake-Sleep (RWS) Training Step

Require: Model’s proposal parameters ϕ and generative parameters θ

Require: Number of particles K

Require: Input sequence comprising x interleaved text and formal tokens

```
1: for  $k = 1, \dots, K$  do
2:   Sample binary submask  $z_k \sim q_\phi(z \mid x)$   $\triangleright$  Particle Sampling
3:    $x_k \leftarrow$  Apply submask  $z_k$  to  $x$ 
4:    $w_k \leftarrow p_\theta(x_k)/q_\phi(z_k \mid x)$ 
5: end for

6:  $\tilde{w}_1, \dots, \tilde{w}_K \leftarrow \text{Softmax}(\log w_1, \dots, \log w_K)$   $\triangleright$  Normalized Weights

7:  $\mathcal{J}_{\text{RWS}} \leftarrow -\sum_k \tilde{w}_k \log q_\phi(z_k \mid x)$   $\triangleright$  Wake- $\phi$  Phase
8: Update  $\phi$  via gradient descent on  $\mathcal{J}_{\text{RWS}}$ 

9: Subsample single particle  $k' \sim \text{Categorical}(\tilde{w}_1, \dots, \tilde{w}_K)$   $\triangleright$  Wake- $\theta$  Phase
10:  $\mathcal{J}_{\text{RWS}} \leftarrow -\log p_\theta(x_{k'})$ 
11: Update  $\theta$  via gradient descent on  $\mathcal{J}_{\text{RWS}}$ 
```

suffer from signal collapse near local optima. In contrast, RWS prevents this collapse by maintaining a constant total gradient weight of 1, effectively acting as a trust-region regularization of OVIS.

5 WoFT Improves Surface Modeling

In this section, we empirically evaluate whether Weave-of-Formal-Thought fine-tuning via RWS improves the base language model’s ability to model surface tokens. We first detail the experimental setup, including the tokenization strategy and the optimization hyperparameters. We then present our preliminary results comparing the generative performance of our approach against standard supervised fine-tuning baselines.

5.1 Experimental setup

We conduct our experiments on Python code generation using StarCoder2-3B [16] as the base architecture. Our training dataset consists of a 15,000-sequence subset sampled from The Stack v2 [16], specifically targeting Python files between 1 and 100,000 bytes. To process these files into formal sequences, we rely on StarCoder2-3B’s tokenizer for the underlying text. We parse the raw Python source code using Tree-sitter to generate an abstract syntax tree (AST), and then traverse this AST, interleaving the text tokens with newly introduced formal tokens (grammar rule names) that represent the syntactic structure. The formal tokens are placed using the prefix order. This extended formal vocabulary is appended directly to the base tokenizer’s vocabulary. Finally, the sequences are processed with a maximum length of 16,000 tokens.

To efficiently accommodate the newly introduced formal grammar vocabulary and enable the model to internalize syntactic structure, we employ low-rank adaptation (LoRA) [37] with a rank of $r = 32$. The adapters are applied comprehensively across all major linear modules, including the attention projections (q_proj , k_proj , v_proj , o_proj), feed-forward layers (c_fc , c_proj), and crucially, the embedding and language modeling heads ($embed_tokens$, lm_head).

All models are optimized using AdamW [38] with a learning rate of 3×10^{-4} , a weight decay of 0.01, and a maximum gradient norm of 1.0. Training is performed on an NVIDIA A40 GPU in `bf16` precision, leveraging PyTorch’s [39] Scaled Dot-Product Attention (SDPA) [40, 41] and gradient checkpointing [42] to maximize memory efficiency. For the WoFT model trained via the reweighted wake-sleep algorithm, we use $K = 4$ independent particles (rollouts) per sequence to construct the variance-reduced gradient estimators. Our implementation is built upon the Hugging Face transformers [43], datasets [44] and peft [45] libraries. The complete training and evaluation pipeline is open-source and available at <https://github.com/alexbouayad/formal>.

5.2 Preliminary results

To evaluate the performance of Weave-of-Formal-Thought (WoFT) fine-tuning, and to assess the efficacy of our training method, we compare the generative performance of our model trained via the reweighted wake-sleep (RWS) algorithm against two standard supervised fine-tuning (SFT) baselines. All models are trained to optimize the standard next-token prediction objective, which we measure via the per-token cross-entropy loss.

To isolate the effect of our discrete latent variable formulation from the mere presence of syntactic data, we evaluate two baselines. First, a *Text SFT* baseline is trained using standard autoregressive SFT exclusively on the original surface text, without any formal grammar tokens. Second, a *Text+Formal SFT* baseline is trained on surface text interleaved with formal grammar tokens using again standard autoregressive SFT. This second baseline evaluates whether simply exposing the model to the flattened syntactic derivation is sufficient to improve performance, or whether our training method offers additional benefits.

Following standard practice in modern large-scale language modeling, our models are fine-tuned for exactly one epoch over a deduplicated corpus. In this single-epoch regime, the model never encounters the same sequence twice, effectively precluding sequence-level overfitting. Consequently, the online training loss provides a continuous, unbiased estimator of out-of-sample generalization error and faithfully reflects the model’s capacity to fit the true data distribution. We therefore report the smoothed online training loss for all runs, computed as a running average over 250 consecutive training steps.

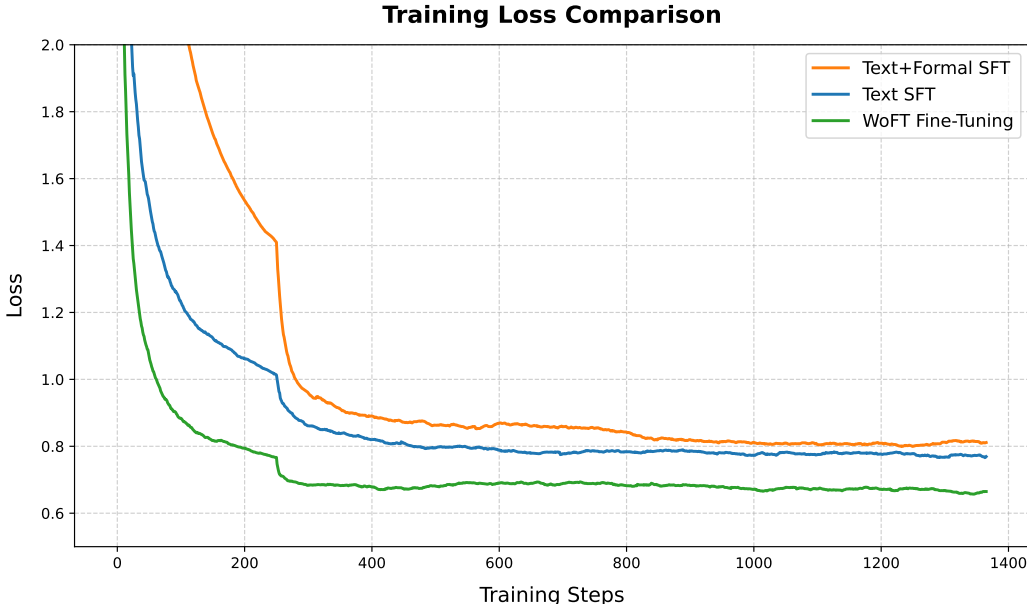


Figure 1: Smoothed online training loss (per-text-token cross-entropy) across the three fine-tuning methods, depicted as a running average over 250 consecutive steps. WoFT fine-tuning via our RWS algorithm achieves a substantially lower surface-token cross-entropy than both the text-only SFT and Formal SFT baselines.

As shown in Figure 1, training WoFT with the RWS objective yields a substantial reduction in the per-token cross-entropy over the surface tokens. The text-only SFT baseline converges to a cross-entropy loss of approximately 0.77 (where all reported convergence values represent running averages over 250 consecutive steps). Interestingly, the Formal SFT baseline performs slightly worse (converging to ~ 0.82), suggesting that simply forcing the model to deterministically predict the formal tokens via teacher forcing distracts it from accurately modeling the surface distribution. From an information-theoretic perspective, modern pre-trained models already encode deep syntactic structure within their implicit representations. Forcing the model to explicitly articulate every formal

derivation step redundantly unrolls this compressed knowledge, bloating the sequence length and reallocating modeling capacity away from the high-entropy surface tokens towards trivial syntax.

In contrast, WoFT fine-tuning via our RWS algorithm successfully marginalizes over the formal latent space, converging to a significantly lower surface token cross-entropy of 0.66. Because the formal tokens are treated as discrete latent variables, the model is not compelled to emit the entire deterministic trace. Instead, it selectively retains formal derivations only when they effectively compress the subsequent surface text—acting as an adaptive structural “scratchpad” for complex dependencies, while gracefully dropping formal tokens when the base model’s implicit syntax suffices. This represents a roughly 14.3% relative reduction in loss compared to the text-only SFT baseline. This result confirms our hypothesis: enabling the model to learn a discretionary, latent grammar derivation recovers critical structural information that flat, surface-level autoregressive training discards, demonstrating the power of dynamically weaving formal syntax into surface generation.

6 Next Steps and Research Vision

Our training paradigm treats formal syntax derivations as discrete latent variables optimized via amortized variational inference, conceptually similar to variational autoencoders (VAEs). To better balance the exploration of latent syntax paths with exploitation of the generative head, it would be interesting to investigate KL-annealing schedules, as well as alternative state-of-the-art estimators, including OVIS [36] and ARM (Antithetic-REINFORCE-Multi-Sample Gradient for Binary Variables) [34]. In standard settings, the sleep phase of wake-sleep is typically omitted because training the proposal on dreamed data from an untrained generative model causes severe distribution bias and instability. However, because our approach fine-tunes a highly capable pre-trained base model, data sparsity and low-quality dreams are not primary bottlenecks. Future iterations could explore reincorporating the sleep phase as a posterior regularizer.

Weave of Formal Thought provides a distinct structural counterpart to recent reinforcement learning and latent reasoning methods, such as Quiet-STaR [3] and latent-variable chain-of-thought fine-tuning [46]. While those methods allow models to allocate extra compute via unconstrained latent vectors or free-form text, WoFT operates over a grounded, verifiable formal grammar. Our hybrid formulation directly builds upon the core insights of Unsupervised Recurrent Neural Network Grammars (URNNG) [23], which demonstrated that combining supervised syntactic signals with unsupervised latent grammar exploration yields superior language modeling performance compared to either regime in isolation. However, URNNGs rely on an inference procedure that scales cubically, $\mathcal{O}(T^3)$, with respect to the surface sequence length T , severely limiting their applicability to long code documents. In contrast, our shared-backbone architecture is lightweight, maintains linear complexity with respect to the grammar insertions, and is fully amenable to modern GPU batching. These architectural efficiencies justify our selection of StarCoder2-3B [16] as our primary experimental vehicle. The availability of The Stack v2 dataset provides a massive, highly curated, and deduplicated pre-training corpus, allowing us to evaluate latent syntax fine-tuning over diverse, real-world codebases without confounding pre-training distribution artifacts.

To further minimize constrained decoding overhead, we plan to integrate advanced low-level execution mechanisms, including byte-level lexing, trie-based vocabulary masking, and optimized rejection sampling architectures inspired by high-performance generation engines like Guidance [11] and XGrammar [8]. We also aim to advance lazy compilation by incorporating derivative-based parsing for lexing and fusing Earley parsers [17] with our GLR lattice parser. In parallel, we intend to supplement our engine with the highly efficient incremental parsing capacities offered by Tree-sitter, which parsing based on the GLR algorithm can natively support [25]. Finally, we are developing a low-level external stream scanner API to seamlessly accommodate languages requiring arbitrary external state tracking directly over token streams, extending Tree-sitter’s external scanner API to autoregressive generation.

Future empirical work will conduct extensive ablation studies to isolate the mechanisms driving our observed loss reductions. Specifically, we will investigate regimes where full deterministic syntax injection brings no improvement (confirming that teacher-forcing the complete syntax trace provides the same or worse modeling capacity than baseline SFT), as well as evaluating random masking and caching strategies (with and without fixed frequencies) to verify that the gains stem specifically from learned, adaptive latent routing. Crucially, we aim to evaluate our fine-tuned models across standard,

widely established benchmarks for code generation (such as HumanEval and MBPP) to rigorously assess downstream functional correctness and execution accuracy.

Looking forward, we aim to leverage formal grammars as a rigorous test bed for exploring hierarchical, tree-structured generation rather than strictly linear, left-to-right decoding. Because formal non-terminals encapsulate complete syntactic subtrees, they offer a natural mechanism for the structured compression of past generated text, allowing models to maintain long-range structural coherence across extended contexts. Furthermore, while our current architecture operates over discrete grammar tokens, an exciting frontier involves internalizing these representations even further by working directly with continuous formal tokens in a continuous latent space [4]. By freeing the model from discrete token sampling while retaining the structural boundaries of the formal engine, this approach could enable fully fluid, end-to-end differentiable syntactic reasoning.

Ultimately, our findings corroborate and extend the concluding hypothesis of Kim et al. [23]: explicitly modeling syntax assists surface generation. In the modern era of large pre-trained language models, this hypothesis takes on new significance. Rather than using syntax to bootstrap basic fluency, WoFT uses latent syntax to provide an adaptive structural scratchpad that organizes complex generative plans. This aligns closely with established findings in cognitive science and neuroscience, where experimental evidence demonstrates that the human brain constructs hierarchical, a priori grammatical hypotheses during real-time language processing. By internalizing these formal structures as dynamic latent variables, Weave of Formal Thought provides a rigorous foundation for bridging the gap between surface fluency and structural reasoning.

Acknowledgements

The author thanks G-Research for their support and authorization to publish this work. The author is also deeply grateful to Inria Paris for their welcoming hospitality and supportive research environment.

References

- [1] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, brian ichter, Fei Xia, Ed Chi, Quoc V Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, volume 35, pages 24824–24837, 2022.
- [2] Sachin Goyal, Ziwei Ji, Ankit Singh Rawat, Aditya Krishna Menon, Sanjiv Kumar, and Vaishnavh Nagarajan. Think before you speak: Training language models with pause tokens. In *The Twelfth International Conference on Learning Representations*, 2024.
- [3] Eric Zelikman, Georges Harik, Yijia Shao, Varuna Jayasiri, Nick Haber, and Noah D. Goodman. Quiet-STaR: Language Models Can Teach Themselves to Think Before Speaking, 2024.
- [4] Shibo Hao, Sainbayar Sukhbaatar, DiJia Su, Xian Li, Zhiting Hu, Jason E Weston, and Yuandong Tian. Training large language models to reason in a continuous latent space. In *Second Conference on Language Modeling*, 2025.
- [5] Torsten Scholak, Nathan Schucher, and Dzmitry Bahdanau. PICARD: Parsing incrementally for constrained auto-regressive decoding from language models. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 9895–9901, 2021.
- [6] Brandon T. Willard and Rémi Louf. Efficient Guided Generation for Large Language Models, 2023.
- [7] Shubham Ugare, Tarun Suresh, Hangoo Kang, Sasa Misailovic, and Gagandeep Singh. SynCode: LLM generation with grammar augmentation. *Transactions on Machine Learning Research*, 2025.
- [8] Yixin Dong, Charlie F. Ruan, Yaxing Cai, Ziyi Xu, Yilong Zhao, Ruihang Lai, and Tianqi Chen. XGrammar: Flexible and Efficient Structured Generation Engine for Large Language Models. In *Eighth Conference on Machine Learning and Systems*, 2025.

- [9] Linzhang Li, Yixin Dong, Guanjie Wang, Ziyi Xu, Alexander Jiang, and Tianqi Chen. XGrammar-2: Efficient Dynamic Structured Generation Engine for Agentic LLMs, 2026.
- [10] guidance-ai. LLMGuidance: Super-fast Structured Outputs, 2025.
- [11] guidance-ai. Guidance: A guidance language for controlling large language models, 2023.
- [12] Tree-sitter contributors. Tree-sitter: An Incremental Parsing System for Programming Tools, 2026.
- [13] Masaru Tomita. An efficient augmented-context-free parsing algorithm. *Computational Linguistics*, 13:31–46, 1987.
- [14] Jörg Bornschein and Yoshua Bengio. Reweighted wake-sleep. In *3rd International Conference on Learning Representations, ICLR 2015, San Diego, CA, USA, May 7-9, 2015, Conference Track Proceedings*, 2015.
- [15] Yuri Burda, Roger Grosse, and Ruslan Salakhutdinov. Importance Weighted Autoencoders. In *The Fourth International Conference on Learning Representations*, 2016.
- [16] Anton Lozhkov, Raymond Li, Loubna Ben Allal, Federico Cassano, Joel Lamy-Poirier, Nouamane Tazi, Ao Tang, Dmytro Pykhtar, Jiawei Liu, Yuxiang Wei, Tianyang Liu, Max Tian, Denis Kocetkov, Arthur Zucker, Younes Belkada, Zijian Wang, Qian Liu, Dmitry Abulkhanov, Indraneil Paul, Zhuang Li, Wen-Ding Li, Megan Risdal, Jia Li, Jian Zhu, Terry Yue Zhuo, Evgenii Zheltonozhskii, Nii Osae Osae Dade, Wenhao Yu, Lucas Krauß, Naman Jain, Yixuan Su, Xuanli He, Manan Dey, Edoardo Abati, Yekun Chai, Niklas Muennighoff, Xiangru Tang, Muhtasham Oblokulov, Christopher Akiki, Marc Marone, Chenghao Mou, Mayank Mishra, Alex Gu, Binyuan Hui, Tri Dao, Armel Zebaze, Olivier Dehaene, Nicolas Patry, Canwen Xu, Julian McAuley, Han Hu, Torsten Scholak, Sebastien Paquet, Jennifer Robinson, Carolyn Jane Anderson, Nicolas Chapados, Mostofa Patwary, Nima Tajbakhsh, Yacine Jernite, Carlos Muñoz Ferrandis, Lingming Zhang, Sean Hughes, Thomas Wolf, Arjun Guha, Leandro von Werra, and Harm de Vries. StarCoder2 and The Stack v2: The Next Generation, 2024.
- [17] Jay Earley. An efficient context-free parsing algorithm. *Communications of The Acm*, 13(2):94–102, 1970.
- [18] Luca Beurer-Kellner, Marc Fischer, and Martin Vechev. Guiding LLMs the right way: Fast, non-invasive constrained generation. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 3658–3673, 2024.
- [19] Ciprian Chelba and Frederick Jelinek. Exploiting syntactic structure for language modeling. In *36th Annual Meeting of the Association for Computational Linguistics and 17th International Conference on Computational Linguistics, Volume 1*, pages 225–231, 1998.
- [20] Brian Roark. Probabilistic top-down parsing and language modeling. *Computational Linguistics*, 27(2):249–276, 2001.
- [21] Eugene Charniak. Immediate-head parsing for language models. In *Proceedings of the 39th Annual Meeting of the Association for Computational Linguistics*, pages 124–131, 2001.
- [22] Chris Dyer, Adhiguna Kuncoro, Miguel Ballesteros, and Noah A. Smith. Recurrent neural network grammars. In *Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 199–209, 2016.
- [23] Yoon Kim, Alexander Rush, Lei Yu, Adhiguna Kuncoro, Chris Dyer, and Gábor Melis. Un-supervised recurrent neural network grammars. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 1105–1117, 2019.
- [24] Donald E. Knuth. On the translation of languages from left to right. *Information and Control*, 8(6):607–639, 1965.

- [25] Tim Allen Wagner. *Practical Algorithms for Incremental Software Development Environments*. PhD thesis, University of California, Berkeley, 1997.
- [26] Yoshua Bengio, Nicholas Léonard, and Aaron Courville. Estimating or Propagating Gradients Through Stochastic Neurons for Conditional Computation, 2013.
- [27] Eric Jang, Shixiang Gu, and Ben Poole. Categorical Reparameterization with Gumbel-Softmax, 2017.
- [28] Chris J. Maddison, Andriy Mnih, and Yee Whye Teh. The Concrete Distribution: A Continuous Relaxation of Discrete Random Variables, 2017.
- [29] John Schulman, Nicolas Heess, Theophane Weber, and Pieter Abbeel. Gradient Estimation Using Stochastic Computation Graphs, 2016.
- [30] Tuan Anh Le, Adam R. Kosiorek, N. Siddharth, Yee Whye Teh, and Frank Wood. Revisiting Reweighted Wake-Sleep for Models with Stochastic Control Flow, 2019.
- [31] Romain Lopez, Pierre Boyeau, Nir Yosef, Michael I. Jordan, and Jeffrey Regier. Decision-Making with Auto-Encoding Variational Bayes, 2020.
- [32] Andriy Mnih and Danilo J. Rezende. Variational inference for Monte Carlo objectives, 2016.
- [33] Tom Rainforth, Adam Kosiorek, Tuan Anh Le, Chris Maddison, Maximilian Igl, Frank Wood, and Yee Whye Teh. Tighter variational bounds are not necessarily better. In *Proceedings of the 35th International Conference on Machine Learning*, volume 80 of *Proceedings of Machine Learning Research*, pages 4277–4285, 2018.
- [34] Mingzhang Yin and Mingyuan Zhou. ARM: Augment-REINFORCE-merge gradient for stochastic binary networks. In *International Conference on Learning Representations*, 2019.
- [35] Zhe Dong, Andriy Mnih, and George Tucker. DisARM: An antithetic gradient estimator for binary latent variables. In *Advances in Neural Information Processing Systems*, volume 33, pages 18637–18647, 2020.
- [36] Valentin Liévin, Andrea Dittadi, Anders Christensen, and Ole Winther. Optimal variance control of the score-function gradient estimator for importance-weighted bounds. In *Advances in Neural Information Processing Systems*, volume 33, pages 16591–16602, 2020.
- [37] Edward J Hu, yelong shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.
- [38] Ilya Loshchilov and Frank Hutter. Decoupled Weight Decay Regularization, 2019.
- [39] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zach DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. PyTorch: An Imperative Style, High-Performance Deep Learning Library, 2019.
- [40] Tri Dao, Dan Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems*, volume 35, pages 16344–16359, 2022.
- [41] Tri Dao. FlashAttention-2: Faster attention with better parallelism and work partitioning. In *The Twelfth International Conference on Learning Representations*, 2024.
- [42] Tianqi Chen, Bing Xu, Chiyuan Zhang, and Carlos Guestrin. Training Deep Nets with Sublinear Memory Cost. *CoRR*, abs/1604.06174, 2016.

- [43] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander M. Rush. HuggingFace’s Transformers: State-of-the-art Natural Language Processing, 2020.
- [44] Quentin Lhoest, Albert Villanova del Moral, Yacine Jernite, Abhishek Thakur, Patrick von Platen, Suraj Patil, Julien Chaumond, Mariama Drame, Julien Plu, Lewis Tunstall, Joe Davison, Mario Šaško, Gunjan Chhablani, Bhavitvya Malik, Simon Brandeis, Teven Le Scao, Victor Sanh, Canwen Xu, Nicolas Patry, Angelina McMillan-Major, Philipp Schmid, Sylvain Gugger, Clément Delangue, Théo Matussi re, Lysandre Debut, Stas Bekman, Pierric Cistac, Thibault Goehringer, Victor Mustar, Fran ois Lagunas, Alexander M. Rush, and Thomas Wolf. Datasets: A Community Library for Natural Language Processing, 2021.
- [45] Lingling Xu, Haoran Xie, Si-Zhao Joe Qin, Xiaohui Tao, and Fu Lee Wang. Parameter-Efficient Fine-Tuning Methods for Pretrained Language Models: A Critical Review and Assessment, 2023.
- [46] Du Phan, Matthew D. Hoffman, David Dohan, Sholto Douglas, Tuan Anh Le, Aaron Parisi, Pavel Sountsov, Charles Sutton, Sharad Vikram, and Rif A. Saurous. Training Chain-of-Thought via Latent-Variable Inference, 2023.